APA: Adaptive-Precision Attention

Exact FP16 for the Few Tiles That Matter, FP4 for the Rest

Raphael Friedmann
https://github.com/kekzl/apa

October 2026, APA 0.7.0

Abstract

In long-context causal prefill, the softmax mass of a query row concentrates in a small number of key tiles. APA is a causal flash-attention prefill kernel for the RTX 5090 (sm_120a) that exploits this: a first pass runs block-scaled FP4 attention (`mma.sync kind::mxf4nvf4`) over every 64-key tile and flags a tile as *hot* when its share of the running row sum exceeds a threshold ε. Hot tiles skip the FP4 PV product and are recomputed by a second pass in exact FP16; the two partial results are merged by a log-sum-exp. Both passes run in one fused launch with persistent pass-2 workers. A hot tile must not set the reference maximum of the FP4 accumulation: when an attention sink does, the remaining probabilities fall below the FP4 scale range and round to zero. Keeping the cold frame at most 2^{32} below hot maxima raises the minimum cosine similarity over all 49152 (row, head) pairs of the worst of six Llama-3.2-3B attention dumps from -0.638155 to 0.841302 and removes all 899 pairs with negative cosine (six dumps), at unchanged speed. The rows that remain worst lose their accuracy to FP4 rounding of Q, not of K, P or V (host emulation per error source); a second FP4 term of Q, UE4M3 instead of power-of-two P scales and block scales of least squared error remove all 137 pairs below cosine 0.9 on twelve dumps and raise the worst minimum from 0.773144 to 0.984284, at 6.7 to 25.4 % more attention time. At $\varepsilon = 0.005$ and 122880 keys, 6.0 to 10.3 % of active tiles are hot and the attention kernel of APA 0.3.0 runs 2.50× faster than the same kernel with every tile exact (2.196× with the 0.730 ms quantization step included); the speedup grows with context from 1.30× at 8192 keys. Integrated into the imp inference engine (version 0.2.0 of the kernel), dense prefill of a 106451-token prompt drops from 11659.35 ms (FlashAttention-2) to 6433.23 ms (1.81×), with perplexity below FlashAttention-2 with FP16 or FP32 accumulation on Llama-3.2-3B and Qwen3-8B. The kernel is deterministic (bitwise identical across reruns), supports GQA, paged KV caches and chunked prefill with an incremental tile cache, and is released as a header-only CUDA library under MPL-2.0.

1 Introduction

Prefill cost of long prompts is dominated by attention: for a prompt of length N the work is $O(N^2 d)$ while the rest of the model is $O(Nd^2)$. FlashAttention [2, 1, 10] removes the memory bottleneck, but the arithmetic remains. Two families of methods attack it. *Low-precision* attention [16, 14, 15] computes $QK^\top$ and PV in INT8, FP8 or FP4 with smoothing and fine-grained scales; it keeps every key but adds error to every score. *Sparse* attention [12, 5, 11] skips keys predicted to carry little mass; it is exact on the kept keys and wrong on the skipped ones.

APA combines both at tile granularity and lets the data decide the split per query block. Every tile is computed cheaply in FP4, so no key is dropped. The few tiles that carry a large share of the row's softmax mass, where FP4 error would dominate the output, are routed to an exact FP16 pass. The decision is made online inside the FP4 pass from quantities the pass computes anyway, so it costs one extra small MMA per tile.

Contributions:

- A two-pass attention scheme with an online hot-tile test whose cold verdicts are monotone safe: the running row sum only grows, so a tile judged cold stays below ε of the final row sum, both as estimated in FP4 (Section 3.2).
- A pass-1 tile order (sink tile first, then the diagonal backwards) that puts high-mass tiles into the running sum early, so the test judges middle tiles against a near-final denominator.
- A cold reference frame decoupled from hot tiles, with a diagnosis of the failure it removes: rows whose output lives in the low-mass tail lost that tail to FP4 underflow next to a dominant sink (Sections 3.2 and 4.1).
- An FP4 pass on sm_120 block-scaled MMA with mean-centred K, per-16-key power-of-two P scales folded into the exponent, a lazy row maximum, and row sums obtained from the same quantized P via an MMA against a ones tile.
- A single fused launch: pass-1 CTAs followed by persistent pass-2 workers that pull work from ticket

counters and wait on per-query-block release flags, deadlock-free by ticket order.

- An error-source analysis of the remaining worst rows (FP4 rounding of Q, then of P, at short context of K), fixed by a two-term FP4 Q, UE4M3 P scales and block scales of least squared error (Section 4.3).
- Deterministic statistics and an incremental tile cache for chunked prefill over paged KV.

2 Background

Online softmax. For a query row q and keys k_j, flash attention keeps a running maximum m, row sum $\ell = \sum_j e^{s_j - m}$ and unnormalized output $o = \sum_j e^{s_j - m} v_j$, rescaling ℓ and o when m grows [7, 8]. Two partial states (m_1, ℓ_1, o_1) and (m_2, ℓ_2, o_2) over disjoint key sets merge exactly:

$$M = \max(m_1, m_2), \quad w_i = e^{m_i - M}, \quad O = \frac{w_1 o_1 + w_2 o_2}{w_1 \ell_1 + w_2 \ell_2}. \tag{1}$$

APA uses this to merge the FP4 result over cold tiles with the FP16 result over hot tiles. All exponentials run in the $\log_2$ domain (`ex2.approx`).

Block-scaled FP4 on sm_120. The RTX 5090 executes block-scaled MMA (`mma.sync`, `kind::mxf4nvf4`, m16n8k64) with E2M1 operands and one UE4M3 scale per 16 elements along k (NVFP4, a variant of the OCP microscaling formats [9]). E2M1 represents $\{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$; UE4M3 has a maximum of 448. APA additionally uses a global FP32 scale per head so that $|x|_{\max}$ maps to $6 \cdot 448 = 2688$.

3 Method

APA computes causal attention for head dim $d = 128$ or $d = 256$ with GQA (H query heads, H_{kv} KV heads, $G = H/H_{kv}$). Keys are split into tiles of $B_{kv} = 64$. Query rows of the G heads sharing a KV head are *packed*: packed row r is position $\lfloor r/G \rfloor$ of query head $h_{kv}G + (r \bmod G)$, so one CTA of 12 warps covers $12 \cdot 16 = 192$ packed rows (8 warps, 128 rows at $d = 256$) and loads each KV tile once for all G heads.

3.1 Preparation

A prep stage converts FP16 inputs into the FP4 layout (Table 1). Stats and KV quant each read all of K and V and run at 87 and 89 % of DRAM bandwidth; sampling the stats cuts prep from 0.733 to 0.482 ms at 122880 keys and from 0.162 to 0.138 ms at 32768 keys. Over all pairs of six dumps the mean cosine moves by $-8.6 \cdot 10^{-5}$ to $+4.8 \cdot 10^{-5}$, as much as rounding the exact head scales to powers of two alone ($-1.1 \cdot 10^{-4}$ to $+4.5 \cdot 10^{-5}$). The times and accuracies in Section 4 use exact stats (0.3.0).

Step	Output
stats	per (batch, KV head): K column sums per chunk of 16 tiles, K and V absolute maxima; every s-th chunk only, at least 16 chunks
finalize	K mean $\bar{k}$ (chunks summed in order), head scales g_k, g_v: 4× the sampled maxima (1× without sampling), rounded up to powers of two
KV quant	per tile a 9216 B blob: K (mean-centred) and $V^\top$ in E2M1, one UE4M3 scale per 16 values (least squared error, see Block scales); a block beyond the UE4M3 range sets a flag that reruns stats over all keys and KV quant, gated on the device
Q quant	packed Q rows in E2M1 with UE4M3 scales; $q \cdot \bar{k}$ kept for pass 2

Table 1: Prep kernels. Blob = K 4096 B, $V^\top$ 4096 B, K scales 512 B, V scales 512 B, laid out for conflict-free `ldmatrix` (XOR swizzle) and for one bulk copy per tile.

1. $S \leftarrow \mathrm{MMA}_{\mathrm{fp4}}(Q, K_t)$; apply causal / tail mask
2. candidate max $c \leftarrow m$, or the row max if a score exceeds $m + \tau$
3. per 16-key group: power-of-two scale 2^k, $P' \leftarrow 2^{sc' + b}$ in frame c, $P' \le 6$
4. $\ell_t \leftarrow \mathrm{MMA}_{\mathrm{fp4}}(P', \mathbf{1})$ tile row sums, frame c
5. if any row has $\ell_t > \varepsilon\,(2^{m-c}\lambda + \ell_t)$: hot
   - $f \leftarrow \max(m,\, c - \delta)$; move o, ℓ_{cold}, λ to frame f
   - $\lambda \leftarrow \lambda + 2^{c-f}\ell_t$; set bit t in the hot mask, stop
6. cold: move o, ℓ_{cold}, λ to frame c; $\lambda \leftarrow \lambda + \ell_t$
7. $\ell_{\mathrm{cold}} \leftarrow \ell_{\mathrm{cold}} + \ell_t$; $o \leftarrow o + \mathrm{MMA}_{\mathrm{fp4}}(P', V_t)$

Figure 1: Pass 1, one tile t of one warp (c': score scale, $\delta = 32$, `APA_HOT_DROP`).

Mean-centred K. Subtracting the per-channel key mean, $s_j = q \cdot (k_j - \bar{k})$, shifts every score of a row by the same constant $q \cdot \bar{k}$ and leaves the softmax unchanged [16]. It removes the shared channel offset that would otherwise consume FP4 range. Pass 2 computes exact scores $q \cdot k_j$ from the original FP16 keys and subtracts $q \cdot \bar{k}$, so both passes share one logit frame and Eq. (1) applies.

3.2 Pass 1: FP4 attention with a hot-tile test

Each warp owns 16 packed rows; a CTA of 12 warps streams the tile blobs through a 6-stage shared-memory ring filled by `cp.async.bulk` with mbarrier completion (thread 0 is the producer). Per tile and warp:

P quantization. The probabilities of a 16-key group are scaled by $1536/s$, where s is the smallest UE4M3 value

with $s \geq 256 \cdot \max_{\text{group}} P$: with u the required $\log_2 s$, $m = \lceil 8(2^{u-\lfloor u \rfloor} - 1) \rceil$ and $s = 2^{\lfloor u \rfloor}(1 + m/8)$ (3 mantissa bits; $m = 8$ carries into the exponent). The quantized values satisfy $P' \leq 6$, and the group maximum lands near 6 instead of anywhere in $(3, 6]$ as with a power of two (0.4.0). The shift $\lfloor u \rfloor + \log_2(1 + m/8)$ is folded into the bias of the exponent, so `ex2` directly produces P'; the global factor 1536 is divided out in the epilogue.

Two-term Q. Q is quantized twice: Q_1 as in Table 1 and Q_2, the E2M1 residual $Q - Q_1$ in the same row scale, so $S = Q_1 K^\top + Q_2 K^\top$ accumulates in one fragment with the same K operands, at 16 more OMMAs per tile. Section 4.3 shows why: FP4 rounding of Q, not of K, P or V, decided the worst rows; with two terms, P became the largest error source, which the UE4M3 scale above addresses.

Block scales. Every block of 16 values the prep quantizes (Q_1, Q_2, K, $V^\top$) takes the UE4M3 scale code among $c_0 + \{0, \pm 1, -2, 3, 4, 5, 6\}$, c_0 the code nearest to $\max |x|/6$, with the least squared error after an E2M1 round trip. The nearest code can round the scale down, which clips the block maximum at 6, or up, which wastes part of the grid. At short context FP4 rounding of K decides the worst rows (Section 4.3). The error per candidate comes from x in f16x2 and one F2FP conversion each way (`cvt.rn.satfinite.e2m1x2.f16x2`): prep for 2048 queries on 26624 keys takes 0.170 instead of 0.137 ms, at 122880 keys 0.501 instead of 0.492 ms; attention time is unchanged.

Lazy maximum. The running maximum m ($\log_2$ domain) is updated, with a full quad reduction and a rescale of o, ℓ_{cold} and λ, only when some score exceeds $m + \tau$ with $\tau = 4$. The bias absorbs the lag, so $P' \leq 6$ still holds; most tiles skip the rescale.

Row sums from the quantized P. Instead of summing FP32 probabilities, pass 1 multiplies the packed P' fragment by a tile of E2M1 ones. The resulting row sums are those of the values that enter PV, so numerator and denominator of the cold partial carry the same quantization.

The hot test. λ accumulates the sums of *all* tiles seen so far, hot ones included, while ℓ_{cold} and o accumulate only cold ones. A tile is hot for the warp if, for any of its 16 rows, its row sum exceeds $\varepsilon\lambda$. The test is warp-uniform (`__any_sync`), so the hot mask has one bit per (warp, tile).

Proposition. Let ℓ_t be pass 1's FP4 row sum of tile t, λ_t the running sum after tile t and λ_∞ the final sum, all in the frame of the final maximum. Every tile t that pass 1 marks cold satisfies $\ell_t \leq \varepsilon\lambda_\infty$, and the cold tiles together carry at most $n_c\,\varepsilon$ of λ_∞, where n_c is their number. The statement is about the FP4 estimates; the FP32 share of a cold tile can exceed ε (Section 4.1: up to 0.0118 at $\varepsilon = 0.005$).

Proof. A rescale multiplies ℓ_t, λ and all later sums by the same factor, so shares are frame-invariant. Sums of exponentials are non-negative, hence $\lambda_t \leq \lambda_\infty$ and $\ell_t \leq \varepsilon\lambda_t \leq \varepsilon\lambda_\infty$. Summing over cold tiles gives the second bound. □

The per-tile bound is what the test guarantees; the total bound $n_c\varepsilon$ is loose (at 122880 keys, n_c can approach 1920). A hot verdict can be conservative; it costs pass-2 work, not accuracy. The FP4 error of the cold part is bounded only through the per-tile share, which is the mechanism behind the worst-case rows in Section 5.

Cold frame. The frame m sets the scale of every later P'. The per-group exponent k is clamped at -6 so that 2^k stays a normal UE4M3 value; a group whose maximum lies more than about 2^{-10} below the frame therefore quantizes to values under 0.25 and rounds to E2M1 zero. In APA 0.2.0 the frame was the running maximum over all tiles. The sink tile, visited first and usually hot, then set m, and in rows whose remaining mass is spread thinly (largest cold tile at most 0.0025 of the row) most cold groups vanished (Section 4.1). Hot tiles do not need m: their P' is used only for ℓ_t, computed in the candidate frame c, and pass 2 recomputes them. APA 0.3.0 therefore lets a hot tile lift the frame only to $\max(m, c - \delta)$ with $\delta = 32$; λ absorbs the hot sum as $2^{c-f}\ell_t \leq 2^{32}\ell_t$, well inside FP32 range. A cold tile commits its candidate as before; with $\delta = 0$ the kernel reproduces the 0.2.0 metrics to all six printed digits on six dumps. Frames are row-wise; the proposition above holds unchanged, since all sums still share one frame. What the change guarantees: a cold group is quantized without the -6 clamp if its maximum lies at most 2^{-10} below the frame, and a hot tile moves the frame to at most 2^{-32} of its own maximum, so a hot tile can push a cold group to zero only if that group lies more than 2^{-42} below the hot tile's maximum. Nothing bounds the FP4 error of the cold mass that remains.

Tile order. The denominator λ is small early in the loop, which makes early tiles look hot. Pass 1 therefore visits tile 0 first (the attention sink [12]), then the remaining tiles from the causal diagonal backwards. The tiles that typically carry most mass enter λ first, and the long middle range is judged against a near-final sum.

Epilogue. A warp without hot tiles normalizes and stores its output directly. A warp with hot tiles exports (m, ℓ_{cold}) and the unnormalized cold partial o in FP32 for the merge.

3.3 Pass 2: exact FP16 over hot tiles

A pass-2 CTA takes the 192 packed rows of one pass-1 query block with the same 12 warps. The union of their hot tiles streams from the original FP16 K and V (flat or paged) through a ring of 3 shared-memory stages of one K and one V tile: any warp that finds the next stage released by all warps claims it (atomic CAS) and fills it with `cp.async`, completion signalled on an mbarrier. Every warp passes every union step, so the 12 warps share each load; against three independent 4-warp groups (0.3.0) this cuts attention time by 1.2 to 3.1 % at $\varepsilon = 0.005$ and 0.01 on four dumps, with bitwise equal outputs. Each warp computes only its own hot tiles with `mma.sync.m16n8k16` (f16 inputs, f16 accumulation, the default of imp's FlashAttention-2 path) and an FP32 online softmax in the $\log_2$ domain. At the end each row is merged with its pass-1 partial by Eq. (1) and written to O.

The output accumulator stays in f16 across tiles by default. With few hot tiles this is harmless; with all tiles hot ($\varepsilon < 0$), small late contributions next to a dominant sink round away (Section 4.1). `APA_P2_F32O=1` sums each 16-token k-step in f16 at full HMMA rate and adds it to an FP32 accumulator; the extra 32 registers per thread cause spills in the 384-thread fused kernel.

3.4 Fused launch

Since 0.7.0 both passes run in one persistent launch with one CTA per SM. A CTA takes tickets in order from an atomic counter: per key segment k (see KV window below) the quantization items of segment $k+1$, then pass 1 of segment k on every query block; past the last pass-1 ticket it pulls whole query blocks for pass 2 from a second counter. Every wait (quantized segment, previous segment of the same query block, ring slot reuse, pass 1 before pass 2) is on an earlier ticket held by a running CTA, so the scheme cannot deadlock. Ready flags are set with a release store after a fence and read with acquire semantics. Pass-2 work fills the SMs left idle by pass 1's last wave.

3.5 KV window

The FP4 copy of K and V holds the sink tile plus 2 ring slots of 96 tiles per (batch, KV head) instead of every tile: 13.6 MiB at $H_{kv} = 8$ for any context (135.0 MiB at 122880 keys before). Pass 1 walks the keys in segments (sink, then the diagonal backwards, as before); the row state (o, m, ℓ) crosses segments in global memory, so tile order and every value equal one pass over all tiles and the outputs are bitwise equal. Quantization of segment $k+1$ runs inside the launch while pass 1 works on segment k.

3.6 Sliding window

`Problem::window` w restricts the query at position p to keys $(p - w, p]$. CTAs and warps skip tiles below their window, the lower edge is masked in pass 1 and pass 2, and KV-window segments below the first query's window are neither quantized nor walked. On synthetic inputs with 122880 keys, $w = 4096$ takes 0.804 ms against 6.053 ms causal (7.53×). The window is a template parameter: the kernels without it are SASS-identical to the previous release.

3.7 Head dim 256

At $d = 256$ a CTA has 8 warps and 4 pass-1 stages. $QK^\top$ loads K fragments per `ldmatrix.x4` group (32 instead of 64 registers), PV reads V scales per pair of n8 tiles, and pass 2 streams 32-key half tiles (2 steps per KV tile) so 3 stages still fit in 96 KiB. Adding $d = 256$ leaves the $d = 128$ kernels SASS-identical.

3.8 Determinism

All floating-point reductions run in a fixed order. In 0.1.0 the K column sums used shared and global float atomics; 5 reruns of prep and attention on the three dumps differed in 539072, 1329742 and 950802 output elements. In 0.2.0 each 16-tile chunk sums its columns in registers, then by XOR shuffles, then over warps in order, and the finalize kernel sums chunks in order. Maxima use integer `atomicMax`, which is order-independent. The same 5 reruns now give bitwise identical outputs; prep time fell from 0.865 to 0.731 ms at 122880 keys.

3.9 Paged KV and chunked prefill

Pass 2 reads K/V through a reader abstraction: flat `[B][S][H_kv][d]` or a paged FP16 pool through a block table [6], plus a flat tail for the chunk not yet in the cache. For chunked prefill, `prefill_incremental` keeps a per-layer `KvState` with the quantized tile blobs and statistics; a new chunk quantizes only its new keys. K mean and head scales stay frozen until the context has grown by a factor `KV_RESTAT` (1.125, 2 in 0.3.0); then every tile is requantized. Head scales are widened by a headroom factor so later keys fit, and a fingerprint of 48 key values detects a stale cache and triggers a full requantization.

4 Evaluation

Setup. RTX 5090 (sm_120a), CUDA 13. Standalone runs use attention inputs dumped from Llama-3.2-3B [4] (24 query heads, 8 KV heads, $d = 128$): 2048 query rows against 122880 keys (lc_122880_{0,1,2}) or 32768 keys (lc_32768_{0,1,2}). Accuracy is the cosine similarity against an FP32 reference, in two modes. *All pairs*: every one of the $2048 \cdot 24 = 49152$ (row, head) pairs; we report the mean and minimum of the per-pair cosines and the number of pairs below 0.9 and 0. *Sample*: 512 pairs drawn with a fixed seed (1.04 %); "cos" is then the pooled cosine of all sampled outputs concatenated and "min" the

Quantity	0.2.0, $\varepsilon = 0.01$	0.2.0, $\varepsilon < 0$
row (position, head)	190, 20	214, 20
mass of the top tile	0.916	0.913
hot tiles, their mass	2, 0.920	1892, 1.000
‖hot part of ref‖	0.2527	0.1197
‖cold part of ref‖	0.3634	0
‖ref‖ / ‖APA out‖	0.1670 / 0.2531	0.1197 / 0.1331
$\|\text{ref}\| / \sum_j p_j \|v_j\|$	0.1997	0.1479
cosine	−0.4446	0.8766

Table 2: Worst sampled pair of lc_122880_2 (sample mode). Left: FP4 cold part. Right: every tile exact, f16 output accumulation in pass 2.

sample minimum, which overstates the all-pairs minimum on every dump (Table 3). Tables state their mode. Times are the minimum over 5 repetitions of the mean of 5 launches (CUDA events) of the fused attention kernel; the quantization step (prep) is timed separately and excluded unless stated. Absolute times drift by up to 10 % between runs on this GPU (lc_122880_1 at $\varepsilon = 0.005$: 4.707 and 5.200 ms in two runs), so every comparison below is taken within one run. Hot share is the fraction of active (warp, tile) pairs flagged hot. The kernel is deterministic, so accuracy figures do not vary between runs.

4.1 Diagnosing the worst rows

In sample mode APA 0.2.0 reached a cosine of −0.444615 on lc_122880_2; over all pairs the minimum is −0.638155, with 789 pairs below 0 (Table 3). A diagnostic mode of the benchmark (`APA_DIAG=1`) decomposes the reference output of the worst sampled pair into the contributions of the tiles pass 1 marked hot and cold (Table 2).

In the left column, the sink tile holds 0.916 of the softmax mass, yet the 8 % spread over the other 1890 tiles carries an output component (0.3634) larger than the whole reference (0.1670): the two parts nearly cancel, and ‖ref‖ is only 0.20 of the probability-weighted mean value norm. The APA output norm 0.2531 equals the hot part alone (0.2527): the cold contribution was lost, the FP4 underflow of Section 3.2. In the right column every tile is exact, yet the cosine is 0.8766: pass 2 accumulates O in f16, and next to the sink each small tile's contribution falls under half an f16 ulp of the accumulator.

All pairs. The full mode (`APA_FULL=1`) adds, for the 20 worst pairs of a run, the FP32 mass of the warp's cold tiles and, in a debug build, pass 1's own m, λ and ℓ_{cold}. All 20 worst pairs of every $\varepsilon = 0.005$ run are on the merge path (their warp has hot tiles). For the 8 worst pairs of lc_122880_2 under 0.2.0 the FP32 cold mass is 0.0962 to 0.1511, the FP4 estimate $\ell_{\text{cold}}/\lambda$ only 0.0069 to 0.0254, with m at 21.94 to 22.68 (the sink's frame): the tail was quantized away. Under 0.3.0 the FP4 estimate is 0.0462 to 0.0977 against FP32 0.0548

	0.2.0			0.3.0		
Dump	min	< 0	ms	mean	min	ms
lc_122880_0	0.634875	0	4.555	0.998512	0.959676	4.490
lc_122880_1	0.925502	0	4.676	0.999717	0.975775	4.759
lc_122880_2	−0.638155	789	4.758	0.998438	0.841302	4.840
lc_32768_0	0.757021	0	2.245	0.999037	0.927517	2.285
lc_32768_1	0.927160	0	1.881	0.999827	0.980919	1.889
lc_32768_2	−0.594803	110	1.856	0.999372	0.969776	1.863

Table 3: Cold frame, $\varepsilon = 0.005$, all 49152 pairs, one run. "mean", "min": per-pair cosines; "< 0": pairs with negative cosine. 0.3.0 has 0 pairs below 0 and 14 below 0.9 (lc_122880_2 only). Sample-mode minima of the same builds: 0.898838, 0.970893, −0.444615, 0.879942, 0.984021, 0.100775 (0.2.0) and 0.969733, 0.997192, 0.944326, 0.967587, 0.996628, 0.987198 (0.3.0).

to 0.0903. Three other explanations were tested and rejected as the cause: cold tiles whose FP32 share exceeds ε occur only on lc_122880_1 (FP32 share up to 0.0118), in pairs with cosine ≥ 0.975775; the worst heads have an ordinary per-channel spread of $|K|$ and $|Q|$ maxima (max/median 3.2 and 3.7 to 4.1, while heads with spreads up to 10.8 are not among the worst); visiting tiles in ascending order (`APA_ORDER=0`) leaves 787 pairs below 0. An exact running maximum (`-DAPA_EXACT_MAX`, $\tau = 0$) widens the unclamped range of a group by 2^4 and already removes the negative pairs of 0.2.0 on lc_122880_2 (minimum 0.251078), which points at the scale clamp. Pure FP4 ($\varepsilon = 10^9$, no pass 2) has 25 pairs below 0 on lc_122880_2, fewer than 0.2.0 with exact hot tiles (789). All runs: AUDIT.md and PERF_LOG.md in the repository.

4.2 Cold frame

Table 3 compares 0.2.0 ($\delta = 0$) with 0.3.0 ($\delta = 32$) in one run. The minimum over all pairs rises on every dump, from −0.638155 to 0.841302 on lc_122880_2 and from −0.594803 to 0.969776 on lc_32768_2, and the 899 negative pairs disappear. Attention times stay within run-to-run noise; hot shares change by at most 0.2 points. Over all pairs (0.3.0, $\varepsilon = 0.005$), the paged path (block size 16) matches the flat one on lc_122880_1 and lc_122880_2 to all six printed digits. Chunked prefill (60 chunks of 2048 queries, last chunk evaluated) with the incremental tile cache and the 0.3.0 restat factor 2 reaches a minimum of 0.679235 and 275 pairs below 0.9 on lc_122880_2, against 0.841302 and 14 for a full requantization per chunk. The cause is the frozen K mean: a full requantization of 120832 keys whose K mean covers only the first 65536, 81920, 98304 or 108544 keys gives 431, 222, 104 and 56 pairs below 0.9 (all keys: 24). Restat at 1.125 instead of 2 times the context cuts the worst chunk against a full requantization from 1452 to 29 pairs below 0.99 (exact stats),

Variant	lc_65536_0	lc_122880_2
kernel (0.4.0)	0.809680	0.888434
emulation, all sources	0.815728	0.890648
P only	0.996408	0.998214
V only	0.999817	0.999897
K only	0.998618	0.999879
Q only	0.823032	0.907227
all, Q two FP4 terms	0.998386	0.998000
all, Q two terms on 64 channels	0.998099	0.996503
all, Q in FP8 E4M3	0.995449	0.993688

Table 4: Mean cosine of the 20 worst pairs ($\varepsilon = 0.005$) when only the named operands of the cold tiles are quantized as in pass 1 (host emulation).

at 182.88 instead of 178.32 ms for 60 chunks. With the sampled stats and power-of-two scales of the prep and an overflow redo, every restat chunk equals the full requantization bitwise (27 of 60 chunks on lc_122880_*); the last chunk reaches 0.821417 and 24 pairs below 0.9 (full requantization: 0.847877 and 13), at 176.53 instead of 183.53 ms. (0.4.0, single-term Q.) In 0.6.1 the last chunk reaches 0.985519 and no pair below 0.9 (full requantization: 0.986760, 0). 5 reruns stay bitwise identical.

4.3 Error source of the worst rows

After 0.4.0 the worst dump is lc_65536_0, outside the tables above: 124 pairs below 0.9 at $\varepsilon = 0.005$. Its 20 worst pairs are diffuse rows of one head: FP32 cold mass 0.2263 to 0.3973 spread over 314 to 454 tiles (90 % of the mass), no cold tile above ε, and a cold output 1.137 to 1.349 times the reference norm. The FP4 estimate of the cold share matches FP32, so no mass is lost; a fallback on a large cold share is not selective, since every warp of the dump has a row with cold share above 0.3. Table 4 emulates on the host one quantization at a time on the cold tiles of the 20 worst pairs (hot tiles exact, as pass 2). The emulation with all sources reproduces the kernel; Q alone accounts for the error, while K, P or V alone keep the mean at or above 0.996408. A second FP4 term of Q removes the error.

With two terms, the worst pairs change. Table 5 repeats the emulation on the 20 worst pairs of an intermediate kernel that kept the second term on 64 channels only (8 instead of 16 extra OMMAs), with its channel order and blocks. P is now the largest source: exact P lifts lc_122880_2 from 0.987731 to 0.998622. The 64-channel variants depend on a channel ranking whose effect changes sign between dumps (by $|q|$, lc_122880_0 falls below a single term); the second term on all 128 channels with UE4M3 P scales is best on all three dumps. On all twelve dumps it is better or equal in mean cosine and in pairs below 0.99 than the 64-channel kernel with ranking by $|q|\, r_k$ and UE4M3 P (1163 against 1640 pairs below 0.99, worst minimum 0.980723 against 0.970867), at −0.4 to +4.7 % attention time (same run). The kernel uses it.

Variant	lc_122880_0	lc_122880_2	lc_65536_0
kernel (64 by $\|q\|$)	0.972181	0.983897	0.984614
emulated, same	0.972530	0.987731	0.984100
single term	0.981784	0.987116	0.976630
two terms on 128 ch	0.985336	0.991681	0.992098
64 by $\|q\|\, r_k$	0.982564	0.990931	0.984532
64 by $\|q\|$, P exact	0.988055	0.998622	0.990676
64 by $\|q\|$, K exact	0.978159	0.987592	0.987526
64 by $\|q\|$, V exact	0.981590	0.989307	0.987325
64 by $\|q\|$, Q exact	0.980647	0.991577	0.989696
64 by $\|q\|$, UE4M3 P	0.973035	0.994771	0.988863
64 by $\|q\|\, r_k$, UE4M3 P	0.983305	0.996541	0.988574
128 ch, UE4M3 P	0.986028	0.997152	0.992603

Table 5: Mean cosine of the 20 worst pairs of the two-term kernel ($\varepsilon = 0.005$), host emulation with the kernel's channel order, blocks and frame-relative P scales. "64 by x": residual on the 64 channels of largest x; $r_k = \mathrm{rms}(k - \bar{k})$ per channel.

	min cos		pairs < 0.99		attn ms	
Dump	0.4.0	now	0.4.0	now	0.4.0	now
lc_122880_0	0.957087	0.984284	1612	887	4.525	5.567
lc_122880_1	0.972723	0.997452	13	0	4.749	5.909
lc_122880_2	0.847877	0.986760	1224	4	4.964	6.223
lc_65536_0	0.773144	0.987245	1296	14	3.034	3.666
lc_65536_1	0.985146	0.997996	6	0	3.168	3.380
lc_65536_2	0.909434	0.992051	370	0	2.903	3.478
lc_32768_0	0.938131	0.993301	503	0	2.391	2.616
lc_32768_1	0.990549	0.998725	0	0	1.865	2.069
lc_32768_2	0.970655	0.992835	170	0	1.926	2.215
ra2dump_59	0.993380	0.999164	0	0	0.168	0.178
ra2dump_78	0.991309	0.997652	0	0	0.149	0.160
ra2dump_87	0.983843	0.998539	2	0	0.296	0.321

Table 6: 0.4.0 (single-term Q, power-of-two P scales, nearest block scales) vs now (0.6.1: two-term Q, UE4M3 P scales, block scales of least squared error), all 49152 pairs, $\varepsilon = 0.005$; times of 0.6.0 (same attention kernel) from one run with 0.4.0, minimum of 2 repetitions. Pairs below 0.9: 137 → 0. ra2dump: 2048 queries at 2048 or 4096 keys.

Table 6 shows the current kernel (with the block scales below). All 137 pairs below 0.9 disappear, the minimum rises on all twelve dumps, the worst from 0.773144 to 0.984284, and pairs below 0.99 fall from 5196 to 905 (9 dumps fewer, 3 at zero in both). Attention time grows by 6.7 to 25.4 % on the nine long dumps (one run): the second Q term doubles the $QK^\top$ OMMAs of pass 1 (pass 1 alone +24.1 to +30.7 %), the UE4M3 P scale adds 5.1 to 5.2 %.

Short context. Scoring every chunk of a chunked prefill against FP32 exposed short contexts: with the first 4096, 6144, 8192 or 10240 keys of lc_122880_2 (the dump's queries at the end of that prefix) the minimum cosine of 0.5.0 is 0.918134, 0.920212, 0.883160 and 0.948890. Emulation of the 20 worst pairs at 8192 keys points at K: exact K lifts their mean cosine from 0.979228 to 0.999511, exact P, V or Q leaves it between 0.979729 and 0.980116. Block scales of least squared error (Section 3.2, Block scales) lift the emulated mean to 0.993929 with K alone.

Candidates	prep ms	pairs < 0.99 lc_122880_0	lc_65536_0	8192 k.
c_0 (0.5.0)	0.492	1115	42	24
c_0, c_4	0.492	1831	75	14
c_0, c_0+1, c_4, c_4+1	0.499	1783	32	17
c_0, $c_0 \pm 1$, c_4	0.499	1175	10	2
$c_0-2..c_0+8$	0.590	892	14	1
$c_0-2..c_0+6$ (0.6.0)	0.530	893	14	1
$c_0-2..c_0+6$, no c_0+2	0.501	887	14	1

Table 7: Block scale candidates (UE4M3 codes; c_0, c_4: nearest to $\max|x|/6$, $\max|x|/4$), least squared error, all 49152 pairs, $\varepsilon = 0.005$; prep time of lc_122880_0. 8192 k.: first 8192 keys of lc_122880_2. Last row kept (0.6.1): error from x in f16x2; the rows above it rank from x in f32.

Dump	ε	hot	pooled	mean	min	< 0.9	ms
122880_1	exact	100 %	0.999649	0.999505	0.974166	0	12.292
	0.001	28.3 %	0.999875	0.999834	0.990713	0	7.794
	0.003	14.1 %	0.999869	0.999808	0.981854	0	5.683
	0.01	5.4 %	0.999650	0.999505	0.955285	0	4.507
	0.02	2.9 %	0.999416	0.999198	0.930447	0	4.184
122880_2	exact	100 %	0.999603	0.993600	0.834726	465	12.049
	0.001	19.0 %	0.999831	0.999274	0.926464	0	7.489
	0.003	9.1 %	0.999643	0.998755	0.857105	11	5.871
	0.01	3.2 %	0.998835	0.997936	0.834358	15	4.844
	0.02	1.7 %	0.997999	0.997372	0.624999	18	4.340
32768_2	exact	100 %	0.999915	0.998839	0.981738	0	3.287
	0.001	47.4 %	0.999946	0.999707	0.985197	0	2.953
	0.003	25.6 %	0.999897	0.999545	0.972839	0	2.243
	0.01	9.5 %	0.999380	0.998994	0.940459	0	1.607
	0.02	4.9 %	0.998617	0.998451	0.928616	0	1.301

Table 8: APA 0.3.0 over ε, dumps lc_..., all 49152 pairs ("pooled": pooled cosine; "mean", "min": per-pair cosines; "< 0.9": pairs below 0.9). Times from one earlier run (sample-mode cosines and hot shares of that run reproduced exactly). "exact": every tile in pass 2 ($\varepsilon < 0$), f16 output accumulation.

Table 7 compares candidate sets in the kernel: a choice between $\max|x|/6$ and $\max|x|/4$ loses on lc_122880_0, a range around the nearest code wins on all three. c_0+7 and c_0+8 were chosen for 7521 of 97.7 million blocks in the emulation of lc_122880_0; dropping c_0+2 costs nothing, dropping c_0-2 or the V search does. Against 0.5.0, pairs below 0.99 fall from 1163 to 905 on the twelve dumps, the worst minimum rises from 0.980723 to 0.984284, and at 8192 keys the minimum rises to 0.987843 (pairs below 0.99: 24 → 1). Mean cosine falls on 3 of 13 runs (lc_122880_2: 0.999833 → 0.999769).

4.4 Threshold and context length

Table 8 shows the trade-off. From $\varepsilon = 0.001$ to 0.02 the hot share falls by a factor of 9.7 to 11.2 and attention time by 1.7 to 2.3×, while the pooled cosine stays above 0.9979. The all-pairs minimum falls faster than the pooled cosine: on lc_122880_2 from 0.926464 ($\varepsilon = 0.001$) to 0.624999 ($\varepsilon = 0.02$), with 0, 11, 15 and 18 pairs below 0.9; the sample minimum stays at 0.944326 from $\varepsilon = 0.01$ on and misses this. lc_122880_1 and lc_32768_2 keep every pair above 0.928. The all-exact run is not the most accurate configuration: over all pairs of lc_122880_2 it reaches a mean of 0.993600, a minimum of 0.834726 and 465 pairs below 0.9, against 0.998438, 0.841302 and 14 for $\varepsilon = 0.005$; its f16 output accumulation is the limit. With `APA_P2_F32O=1` the all-exact run of lc_122880_1 reaches (sample mode) 1.000000 and a minimum of 0.999997 (12.431 to 17.339 ms), while at $\varepsilon = 0.005$ the same option adds 14.5 % (lc_122880_1) and 24.9 % (lc_32768_1) to the attention time for a pooled cosine of 0.999804 instead of 0.999791. It is therefore off by default.

keys	exact ms	APA ms	speedup	hot	exact min	APA min
8192	0.764	0.589	1.30×	36.0 %	0.999591	0.963431
16384	1.557	1.114	1.40×	31.5 %	0.998313	0.950786
32768	3.237	2.025	1.60×	21.3 %	0.991270	0.963669
65536	6.771	3.594	1.88×	17.6 %	0.965983	0.949406
122880	13.023	5.200	2.50×	9.6 %	0.974166	0.975775

Table 9: Context length: first N keys of lc_122880_1 (query rows at $N-2048$ to $N-1$), $\varepsilon = 0.005$, APA 0.3.0. Times from one run, attention kernel only, prep excluded; "min": all-pairs minimum cosine of the all-exact kernel and of APA.

Table 9 shows why the scheme targets long contexts: the hot share falls from 36.0 % at 8192 keys to 9.6 % at 122880, and the attention speedup over the all-exact kernel grows from 1.30 to 2.50× (13.023 / 5.200 ms). With the quantization step (0.730 ms at 122880 keys, needed only by APA) the ratio at 122880 keys is 2.196×. The all-pairs minimum of APA stays between 0.949406 and 0.975775 at every length; at 65536 and 122880 keys the all-exact kernel reaches 0.965983 and 0.974166 (f16 output accumulation).

4.5 End to end

APA is integrated into imp [3]; the config key is `attention.apa_eps`. The imp numbers in this section were measured in the imp repository, not with this repository's benchmark; their logs are not part of APA. Table 10 uses APA 0.3.0 (commit e71624b), default sparse prefill, `apa_min_kv` 8192, mean of 2 runs; FA2 is `apa_eps` 0.

The remaining imp numbers were measured with APA 0.2.0. Llama-3.2-3B Q8_0, 106451-token prompt, `--max-seq-len 108000`, prefill time:

Perplexity. Table 12 uses imp on a chat-turn wrapped corpus, `ppl_corpus_45k_gemma4_turn.txt` (46359 bytes, a different file from Table 10; 13k to 14k tokens), for Llama-3.2-3B and Qwen3-8B [13] in Q8_0, APA 0.2.0 ($\varepsilon = 0.002$ and 0.01 rows: 0.1.0, which 0.2.0 reproduces exactly at $\varepsilon = 0.005$). The two FlashAttention-2 variants differ only in the accumulation type and span 17.3644 to

Model	tokens	FA2 ms	APA $\varepsilon = 0.01$ ms
Llama-3.2-3B	106451	6746.01	4824.90
Qwen3-4B-2507	112280	8802.52	6969.61
Qwen3-8B	33727	3227.40	2848.55
Qwen3-14B Q6_K	33727	5608.78	5191.51

Table 10: imp prefill, APA 0.3.0. Perplexity in the same setup (`imp-cli --perplexity`, chunk 2048, imp's plain `ppl_corpus_45k.txt`, 44994 bytes), FA2 → APA $\varepsilon = 0.01$: Llama-3.2-3B 18.2760 → 18.2836, Qwen3-8B 10.7522 → 10.7549; Qwen3-14B on long8_32k 2.5249 → 2.5248.

Mode	FA2 ms	APA ms	speedup
default (sparse prefill)	6661.95	5241.18	1.27×
dense	11659.35	6433.23	1.81×

Table 11: imp prefill, APA 0.2.0, $\varepsilon = 0.005$. Dense: `sparse_prefill_topk_tokens=0`. Speedup covers the whole prefill, not only attention.

17.3732 (Llama) and 10.7974 to 10.8110 (Qwen). Every APA setting with $\varepsilon > 0$ lies below both; the all-exact run is 0.0009 above FA2 f32 on Llama. Raising ε from 0.002 to 0.01 cuts default-mode prefill from 5915.15 to 4863.56 ms.

5 Limitations

- **Mass is not output.** The hot test bounds each cold tile's FP4 share of the softmax mass, not of the output. The worst pair of 0.3.0 over all pairs (lc_122880_2, position 1370, head 18, cosine 0.841302) has the shape of Table 2: 3 hot tiles, the largest with 0.9101 of the mass, FP32 cold mass 0.0794, and a cold output component 1.918 times the reference norm, so FP4 error on the tail decides the result. A test weighted by the value norm of a tile, or a bound on the accumulated cold mass, would target such rows. In 0.6.1 the worst pair of twelve dumps has cosine 0.984284 (lc_122880_0), which also has the most pairs below 0.99 (887).
- **Chunked prefill.** The incremental tile cache keeps the K mean of its last restat; on lc_122880_2 the pairs below 0.9 grow from 24 to 431 as the context grows to 1.84 times the keys of that mean (1.11 times: 56; single-term Q; with two-term Q the last chunk has none). `KV_RESTAT` 1.125 caps the growth for 2.6 % more time; a per-restat K mean with a per-row score offset would remove it.
- **Exact mode.** The f16 output accumulation of pass 2 limits the all-exact configuration; the FP32 alternative costs 14.5 to 24.9 % at $\varepsilon = 0.005$ through register spills.
- **Granularity.** The hot verdict is per (warp, tile): one row of 16 makes the tile exact for all.

Variant	Llama 3B	Qwen3 8B	prefill ms
FA2, f16 acc.	17.3644	10.7974	6661.95
FA2, f32 acc.	17.3732	10.8110	
APA, all exact	17.3741	10.7868	
APA $\varepsilon = 0.002$	17.3556	10.7855	5915.15
APA $\varepsilon = 0.005$	17.3423	10.7869	5241.18
APA $\varepsilon = 0.01$	17.3385	10.7728	4863.56

Table 12: Perplexity (lower is better) and default-mode prefill time of the 106451-token prompt (Llama), APA 0.2.0 / 0.1.0.

- **Scope.** Head dim 128 or 256 (256 tested on synthetic inputs only), causal, FP16 inputs, sm_120a only.
- **Evidence.** Six dumps from one model; accuracy over all pairs, times from single runs (drift up to 10 %); imp numbers come from the imp repository. A direct comparison with FP4 attention kernels such as SageAttention3 [15] on the same inputs is open.

6 Related work

FlashAttention [2, 1, 10] established IO-aware exact attention; FlashAttention-3 adds FP8 on Hopper. SageAttention [16, 14, 15] quantizes Q, K, P, V to INT8, INT4, FP8 and FP4 with K smoothing by mean subtraction, which APA adopts; it applies one precision to every tile. StreamingLLM [12] keeps sink and recent tokens, MInference [5] predicts sparse patterns for prefill, Quest [11] selects pages by query-aware bounds for decode. These skip keys; APA keeps every key and spends precision instead of dropping work, with the split decided online from the FP4 pass itself.

7 Conclusion

Long-context attention is skewed enough that exact arithmetic on 6 to 10 % of the (warp, tile) pairs at 122880 keys, FP4 on the rest, and an exact log-sum-exp merge make the attention kernel 2.50× faster than an all-exact kernel (2.196× with quantization). The skew that makes this pay also creates the main hazard: a dominant sink can push the informative tail out of the FP4 range. Keeping the cold frame independent of hot tiles removes that failure at unchanged speed. The code is available at `https://github.com/kekzl/apa` under MPL-2.0.

References

[1] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

[2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[3] Raphael Friedmann. imp: LLM inference engine. `https://github.com/kekzl/imp`, 2026.

[4] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[5] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Symposium on Operating Systems Principles (SOSP)*, 2023.

[7] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.

[8] Markus N. Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.

[9] Bita Darvish Rouhani et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[10] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[11] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning (ICML)*, 2024.

[12] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

[13] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[14] Jintao Zhang, Haofeng Huang, Pengle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. SageAttention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization. In *International Conference on Machine Learning (ICML)*, 2025.

[15] Jintao Zhang, Jia Wei, Pengle Zhang, Xiaoming Xu, Haofeng Huang, Haoxu Wang, Kai Jiang, Jun Zhu, and Jianfei Chen. SageAttention3: Microscaling FP4 attention for inference and an exploration of 8-bit training. *arXiv preprint arXiv:2505.11594*, 2025.

[16] Jintao Zhang, Jia Wei, Pengle Zhang, Jun Zhu, and Jianfei Chen. SageAttention: Accurate 8-bit attention for plug-and-play inference acceleration. In *International Conference on Learning Representations (ICLR)*, 2025.